

Computational Methods #4

Problem 1

Plotting functions using the QAT systems requires constructing a generic function object and using it to construct a plottable object which can be added to a plotview.

In the program “`plotFunctions.cpp`” the GENFUNCTION objects are all generated and based on the command line input a switch statement picks which one is used to construct a Plottable with the appropriate X-Y bounds on the PRectF.

Problem 2

From HW3 we made a program to solve for the wavefunction

$$-\frac{\hbar^2}{2m}\psi''(x) + V(x)\psi(x) = E\psi(x)$$

Where the potential and the wavefunction are given by

$$V(x) \begin{cases} 0, & x < -2a \\ V_0, & -2a < x < -a \\ 2V_0, & -a < x < a \\ V_0, & a < x < 2a \\ 0, & x > 2a \end{cases}, \quad \psi(x) = \begin{cases} e^{ikx} + re^{-ikx}, & x < -2a \\ ae^{in_1 kx} + be^{-in_1 kx}, & -2a < x < -a \\ ce^{in_2 kx} + de^{-in_2 kx}, & -a < x < a \\ fe^{in_1 kx} + ge^{-in_1 kx}, & a < x < 2a \\ te^{ikx}, & x > 2a \end{cases}$$

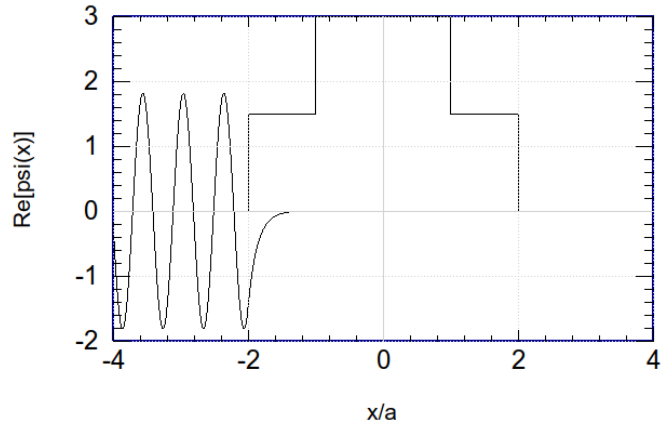
This solution is a vector of coefficients generated by creating a scattering matrix S and inverting it to solve a linear set of equations. Using these coefficients we construct $\psi(x)$. I choose to plot $\text{Re}[\psi(x)]$ to capture the wave nature of the solution. First create a normal function implementing the piecewise nature with an if-else statement.

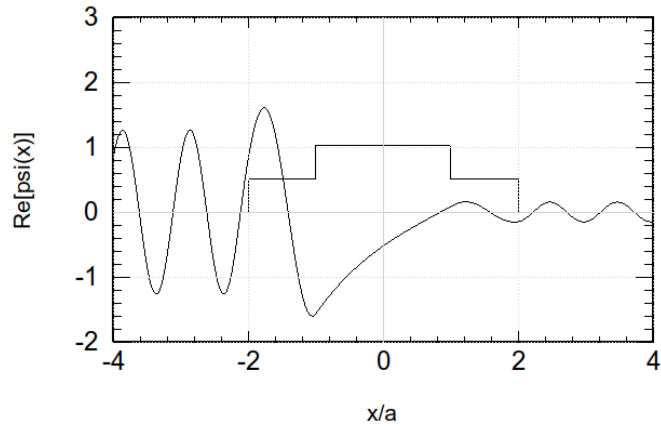
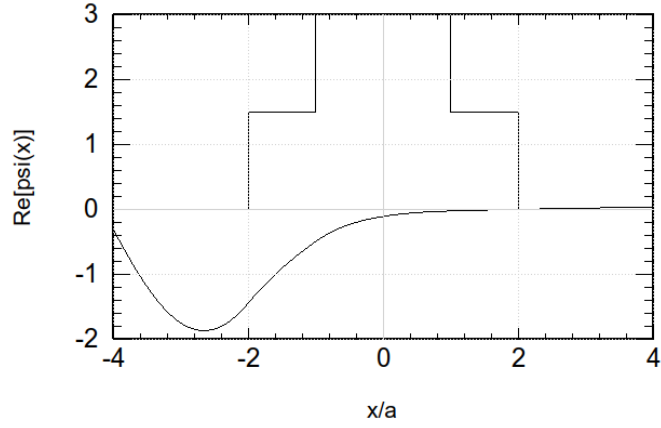
```

double PsiX(double x)
{
std::complex<double> I{0.0, 1.0};
std::complex<double> psix{0};
std::complex<double> n1 = sqrt(std::complex(1.0 - v));
std::complex<double> n2 = sqrt(std::complex(1.0 - 2*v));
if (x <= -2.0)
    {psix = 1.0 * std::exp(I*ka*x)
      + coefficients[0] * std::exp(-I*ka*x);}
else if (x <= -1.0)
    {psix = coefficients[1]*std::exp(I*n1*ka*x)
      + coefficients[2]*std::exp(-I*n1*ka*x);}
else if (x <= 1.0)
    {psix = coefficients[3]*std::exp(I*n2*ka*x)
      + coefficients[4]*std::exp(-I*n2*ka*x);}
else if (x <= 2.0)
    {psix = coefficients[5]*std::exp(I*n1*ka*x)
      + coefficients[6]*std::exp(-I*n1*ka*x);}
else
    {psix = coefficients[7]*std::exp(I*ka*x);}
return psix.real();
}

```

This function can be adapted to an `AbsFunction` through the `F1D Adapter` then it can be used to construct a `PlotFunction1D` object which is added to the `PlotView`. The plots are presented for values $V/E = 1.5$ and $ka = 2\pi/0.6$, $2\pi/6.0$ to represent an input wave with wavelength $\lambda = a$ and $\lambda > a$. I also present the case where $V/E = 0.5$ as it is interesting to see a case with a larger transmission coefficient.



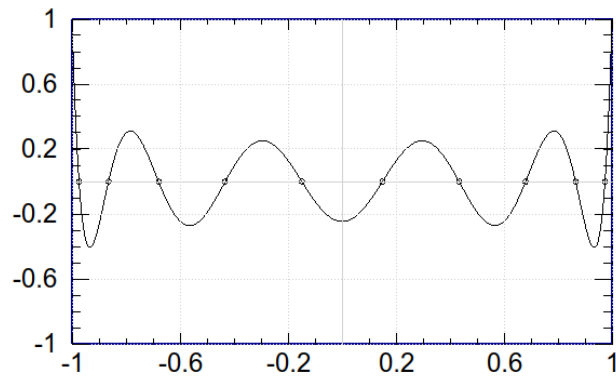
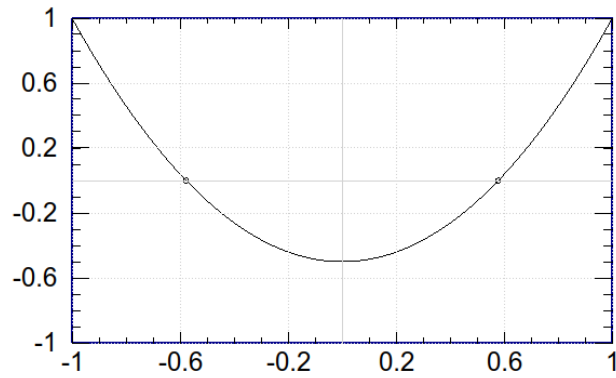


Problem 3

To perform root finding in the QAT system first create the GENFUNCTION whose roots will be found and then create a GENFUNCTION::NewtonRaphson solver object and give it the function. For the initial guess I give it the values

$$x_{l,m} = \left(1 - \frac{1}{8l^2} + \frac{1}{8l^3}\right) \cos\left(\frac{4m-1}{4l+2}\right)$$

To add the points on the plot I create a PlotProfile objects to which you can use the `addPoint(x, y)` method to add all the zeros. I present two of the solutions $P_2(x)$ and $P_{10}(x)$.



Problem 4

This problem is similar to Problem 13 except all the particles have the same mass which means the mass matrix $M = mI$ so the equation of motion is an eigenvalue problem

$$A\ddot{\vec{x}} = \left(\frac{\omega}{\omega_0}\right)^2 \vec{x}$$

Where the matrix A is the dimensionless spring constant matrix K/k

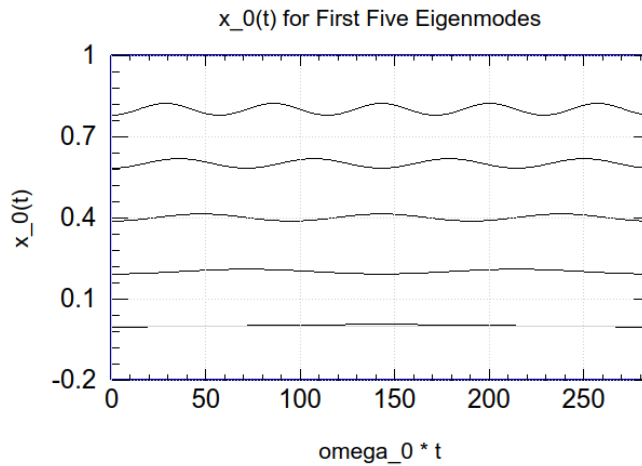
$$A = \frac{1}{2} \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & & \\ & & & \ddots & \\ & & & & 2 & -1 \\ & & & & -1 & 2 \end{pmatrix}$$

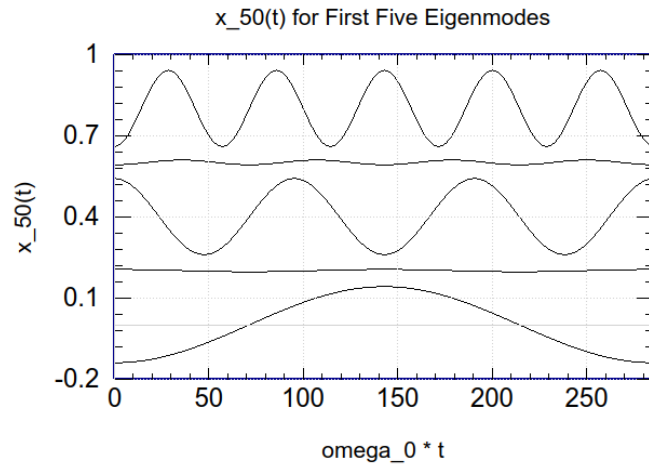
To solve for the eigenmodes simply diagonalize A . The matrix A can be constructed with a for loop where first you initialize the first and last line and then fill in the rest with a loop.

```
Eigen::MatrixXd K(100, 100);
K.setIdentity();

K(0, 1) = -0.5;
K(99, 98) = -0.5;
for(int row = 1; row < 99; row++)
{
    K(row, row-1) = -0.5;
    K(row, row+1) = -0.5;
}
```

After diagonalizing we find the first 5 modes and plot the position of particle i as a function of time for all 5 modes. Here I present the result for $i = 0$ and $i = 50$.





We can see in $x_{50}(t)$ that the odd modes $n = 1, 3$ have small amplitude. This is because these are discretized approximations of the modes of a continuous string and these odd modes have nodes in the middle of the string.